

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 1

З дисципліни: «Обчислювальні методи»

Тема: «Розгортання середовища та порівняльний аналіз мовних моделей»

Варіант № 2

Виконав:

Студент групи АІ-243

Антонович М.Ю.

Перевірив:

ас. Баранов О.М.

Одеса 2026

Мета роботи

Розгорнути локальне середовище для запуску мовних моделей, порівняти локальні та хмарну моделі за швидкістю й якістю, а також дослідити вплив system-повідомлення, temperature і top_p на поведінку моделі.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	323	5
Час, с	19,291	0,404
Токенів/с	17,022	47,774

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
Середній eval_count	1133,00	178,80
Середній час, с	70,606	4,772

Відповіді на питання

1. Куди пішли додаткові токени у міркувальній моделі на простому питанні? У qwen3:4b значна частина токенів пішла у внутрішнє міркування в полі thinking, яке не входить у коротку видиму відповідь. У логу це видно навіть для простого фактичного запиту.

2. На яких задачах міркувальні моделі дають перевагу, а де їх використання не виправдане? Вони корисні в багатокрокових розрахунках, логіці та складному коді. Для простих фактів, короткого форматування або перекладу додаткове міркування часто лише збільшує час і витрати.

3. Чим відрізняється навчання міркувальної моделі від інструктивної? Міркувальні моделі додатково налаштовують на виконання

проміжних кроків перед фінальною відповіддю. Інструктивні моделі більше оптимізовані на пряме виконання команди й швидку відповідь.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Ні. У цьому експерименті qwen3:4b справді дала 5/5 правильних відповідей у задачі на міркування, але більша кількість токенів сама по собі не є показником якості: вона лише показує обсяг виконаної генерації.

5. Як пояснити різницю у швидкості генерації між qwen3:4b та llama3.2:3b? llama3.2:3b менша й на тому самому комп'ютері генерує швидше: 47,774 ток./с проти 17,022 ток./с у qwen3:4b на фактичному запиті. Додатково qwen3:4b витрачає час на внутрішнє міркування.

Висновок за завданням 1

qwen3:4b виявилася значно повільнішою, але на задачі з ручкою та зошитом дала 5 правильних відповідей із 5, тоді як llama3.2:3b — лише 2 із 5. Отже, міркувальна модель виправдана для задач, де потрібні послідовні обчислення, але на простих фактичних запитах її додаткові токени й час є зайвими.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина, символів	Мова	Речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1005	змішана / кострубата	9	Ні
llama3.2:3b	українською, 2 речення	237	укр. + англ. слово	2	Частково
qwen3:4b	лаконічний асистент	193	українська	3	Так
qwen3:4b	українською, 2 речення	174	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	51	325	376
llama3.2:3b	українською, 2	55	84	139

Модель	System	prompt_tokens	completion_tokens	total_tokens
	речення			
qwen3:4b	лаконічний асистент	53	1859	1912
qwen3:4b	українською, 2 речення	62	752	814

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»? За жорсткої інструкції обидві моделі сформували по два речення, але llama3.2:3b вставила англійське слово «contrary». Тому qwen3:4b точніше виконала одночасно обмеження за довжиною і мовою.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку? Так. llama3.2:3b у відповіді з обмеженням до двох речень використала англійське слово, а в іншому запуску також мала змішані та кострубаті конструкції.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому? У llama3.2:3b значення зросло з 51 до 55, а у qwen3:4b — з 53 до 62. Причина в тому, що друге system-повідомлення довше й містить більше вхідних tokenів.

4. Чому в qwen3:4b completion_tokens значно більше, ніж у llama3.2:3b? qwen3:4b є міркувальною моделлю, тому completion_tokens включає не лише видимий текст, а й токени внутрішнього міркування. Через це при короткій видимій відповіді вона витратила 752–1859 вихідних tokenів.

Відповіді на питання (крок 7)

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови? На це впливають розмір моделі, склад навчальних даних та якість instruction-tuning. Малі моделі частіше втрачають окремі обмеження, коли одночасно мають зрозуміти зміст і формат відповіді.

2. Чим system-повідомлення відрізняється від user-повідомлення і чи має воно більший пріоритет? System-повідомлення задає загальні

правила поведінки для запиту й має вищий пріоритет у структурі повідомлень API. Проте це не жорстка програмна гарантія: модель все одно може порушити інструкцію.

3. Чому міркувальна модель витрачає більше вихідних токенів, навіть коли видима відповідь коротка? Вона генерує проміжне міркування перед фінальним текстом. Ці токени враховуються у `completion_tokens`, хоча користувач може бачити лише коротку фінальну відповідь.

4. Що станеться при суперечливих інструкціях у `system` і `user`? Модель має орієнтуватися на `system` як на вищий рівень інструкції, але на практиці виконання може бути нестабільним. Тому критичні вимоги потрібно перевіряти програмно.

5. Чому малі моделі часто гірше дотримуються інструкцій щодо мови? У них менша ємність і зазвичай менше якісних україномовних прикладів у навчанні. Через це вони частіше використовують кальки, змішують мови або втрачають мовне обмеження.

Висновок за завданням 2

System-повідомлення помітно впливає на довжину, мову та формат, але не гарантує повного виконання інструкції. Для надійної роботи в програмі потрібні чіткі обмеження, перевірка результату та, за потреби, повторний запит.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.8-flash
Час відповіді, с	2,914	2,833
completion_tokens	106	190
Довжина відповіді, символів	330	571
Мова відповіді	українська, але зміст помилковий	українська
Фактичних помилок	суттєві	не виявлено

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну? Основний виклик `chat.completions.create()` не змінювався взагалі. Відмінності винесені в `providers.py`, тому `compare.py` лише вибирає назву провайдера.

2. Яка відповідь повніша й точніша за змістом? Хмарна `gemini-3.8-flash` правильно пояснила LIFO для стека та FIFO для черги й навела зрозумілі приклади. `Llama3.2:3b` замість структур даних почала описувати «молекулярну» та «кристалічну» структуру, тобто повністю втратила зміст запиту.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською? Так, фактична помилка була суттєвою: відповідь стосувалася іншої теми. Мова формально була переважно українською, але правильного технічного пояснення не було.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого? У цьому запуску різниця невелика: 2,833 с проти 2,914 с. Хмарні сервери мають значно потужніше спеціалізоване обладнання, яке може компенсувати мережеву затримку.

Відповіді на питання (крок 6)

1. Чому формат OpenAI API став стандартом, який підтримують інші провайдери? Він отримав велику екосистему клієнтів та бібліотек, тому сумісність спрощує міграцію застосунків між провайдерами. `Ollama` та інші сервіси підтримують подібний інтерфейс, щоб існуючий код не доводилося переписувати.

2. Наскільки розмір моделі впливає на якість порівняно з іншими факторами? У моєму експерименті точний розмір `gemini-3.8-flash` не вказувався, тому робити висновок саме про «70B» некоректно. Загалом кількість параметрів важлива, але не менш важливі навчальні дані, архітектура, налаштування та спосіб інференсу.

3. Чому хмарна модель може бути швидшою, попри затримку мережі? Генерація відбувається на спеціалізованому серверному обладнанні, яке може видавати токени значно швидше за локальний ноутбук. На довгих відповідях ця перевага часто перекриває мережеву затримку.

4. Які ризики для приватності виникають при використанні хмарних API? Текст запиту залишає пристрій і обробляється інфраструктурою провайдера. У реальних проєктах мінімізують або анонімізують дані, перевіряють політику зберігання, укладають DPA або використовують локальні моделі для чутливої інформації.

5. Що таке rate limit і як із ним працюють розробники? Це обмеження на кількість запитів або токенів за певний час. Типові рішення — черга запитів, кешування, затримка та повтор із backoff, а при великому навантаженні — перехід на тариф із вищими лімітами.

Висновок за завданням 3

Єдиний OpenAI-сумісний інтерфейс дозволив використовувати той самий код для локальної й хмарної моделі. У цьому тесті хмарна модель була значно точнішою за змістом, а локальна залишається корисною там, де важливі автономність і приватність даних.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

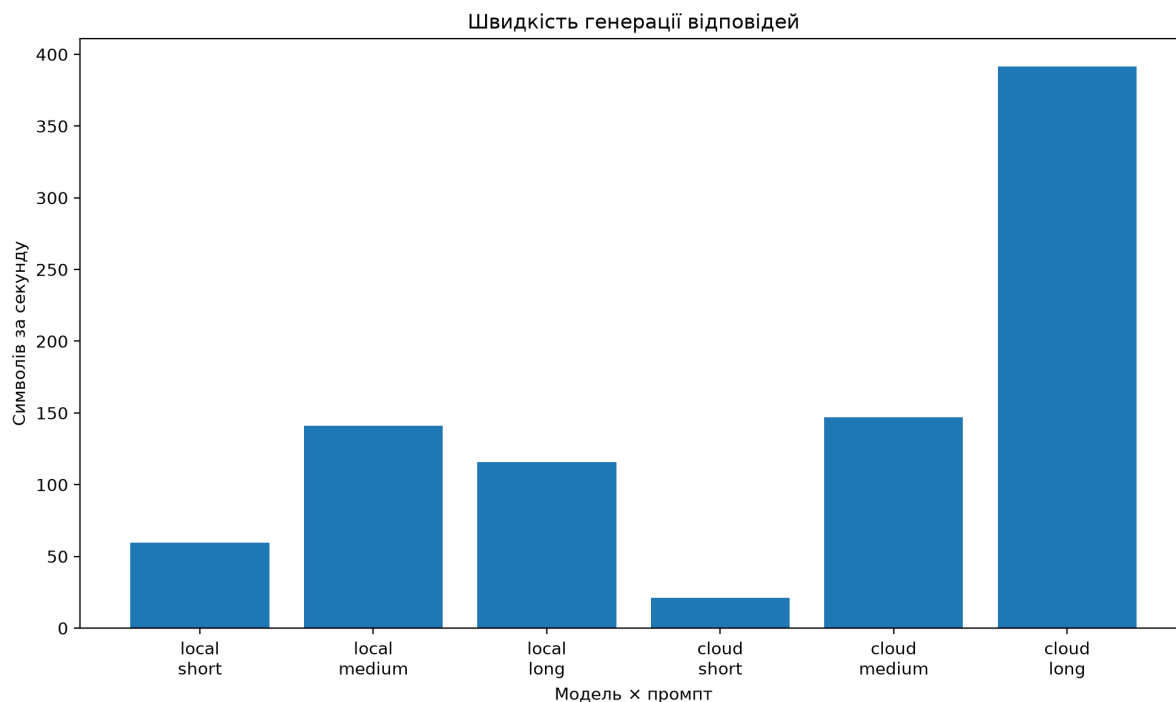
Таблиця 7 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0,045	0,461	59,7
локальна	llama3.2:3b	середній	0,035	2,986	140,9
локальна	llama3.2:3b	довгий	0,064	16,636	115,7
хмарна	gemini-3.8-flash	короткий	1,274	1,275	21,2
хмарна	gemini-3.8-flash	середній	3,322	3,943	147,1
хмарна	gemini-3.8-flash	довгий	5,024	10,614	391,4

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	0,935	0,031
Всього, с	1,451	0,325

Рисунок 1 – Швидкість генерації відповідей



Відповіді на питання (крок 6)

1. У якій моделі кращий TTFT і чому? У всіх трьох промптах кращий TTFT показала локальна llama3.2:3b: 0,035–0,064 с проти 1,274–5,024 с у хмарної моделі. Після прогріву локальна модель уже знаходиться в пам'яті та не має мережевої затримки.

2. Чи збігається найшвидша за TTFT з найшвидшою за символами/с? Ні. На довгому промкті локальна модель має TTFT 0,064 с проти 5,024 с у хмарної, але gemini-3.8-flash генерує 391,4 симв./с проти 115,7 симв./с.

3. Як змінилося співвідношення локальної та хмарної моделі від короткого до довгого промпту? На короткому запиті локальна модель завершила відповідь швидше: 0,461 с проти 1,275 с. На довгому запиті хмарна вже виграла за загальним часом: 10,614 с проти 16,636 с завдяки більшій швидкості генерації.

4. Наскільки різняться три повтори і чи достатньо одного запуску?

Розкид помітний: наприклад, локальний довгий запит тривав 16,636; 16,352 та 25,579 с. Один запуск може потрапити у випадковий викид, тому медіана трьох вимірів дає надійніший результат.

5. Чому холодний старт повільніший за теплий? Після ollama stop модель треба знову завантажити та підготувати в пам'яті. У мене TTFT зменшився з 0,935 с на холодному старті до 0,031 с на теплому.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT і чому його вимірюють окремо? До першого токена система приймає запит, обробляє весь prompt і створює внутрішній стан для генерації; у хмарі додаються мережа та черга сервера. Після цього починається окрема фаза послідовної генерації tokenів.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації? Мережа й планування на сервері збільшують паузу до першого токена. Після старту спеціалізовані серверні прискорювачі можуть генерувати значно швидше за локальний CPU/GPU.

3. Що таке холодний старт? Це перший запит після того, як модель не завантажена в оперативну пам'ять. Перед відповіддю потрібно прочитати ваги моделі з диска й підготувати її до інференсу.

4. Чому беруть медіану кількох запусків? Медіана стійкіша до випадкових стрибків затримки, фонових процесів і мережевих коливань. Це особливо важливо, коли один із трьох запусків значно повільніший за інші.

5. Чому обробка prompt може йти швидше, ніж генерація відповіді такої ж довжини? Prompt обробляється блоками з високим паралелізмом, а відповідь генерується авторегресивно — наступний токен залежить від попереднього. Тому decode є послідовною фазою й гірше паралелізується.

Висновок за завданням 4

Для інтерактивного чат-бота особливо важливий TTFT, бо він визначає, скільки користувач бачить порожній екран. Для фонові обробки великих

текстів важливіші загальна тривалість і пропускна здатність; у моєму тесті хмарна модель програла за TTFT, але на довгому prompt була швидшою за загальним часом.

Завдання 5. Оцінювання якості моделей за рубрикою

Для завдання 1 правильний рік відкриття Київського метрополітену — 1960. Для завдання 3 використано п'ять речень з офіційного Порталу Києва про першу ділянку Святошинсько-Броварської лінії.

Авторитетне джерело для факту: [офіційний портал Києва](#)

Джерело тексту для стиснення: [офіційний портал Києва, містобудівна документація](#)

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	gemini-3.8-flash
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	2	2	1
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	4	9	11

Примітка: якщо бали двох прогонів відрізнялися, для підсумкового бала використано нижчий результат; сама розбіжність окремо показана в таблиці 14.

Таблиця 14 – Розбіжність оцінок між двома прогонами

Завдання	llama3.2:3b	qwen3:4b	gemini-3.8-flash
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	так	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Коротке обґрунтування оцінок

llama3.2:3b двічі назвала 1927 рік замість 1960, але обидва JSON були синтаксично валідними. У стисненні перший прогін втратив частину чисел, код був коректний, логічна задача обидва рази завершилася неправильно, а українська мова була сильно спотворена.

qwen3:4b також не знала правильного року (1985 і 1984), зате обидва JSON парсилися, стиснення зберегло всі числа у двох реченнях, код був коректний, а логічна задача двічі дала правильні 2 рейси. Українська була зрозумілою, але з русизмами й кострубатими формулюваннями.

gemini-3.8-flash стабільно дала 1960, правильний JSON, коректне стиснення, правильну логіку й природну українську. У завданні з кодом функція була правильною, але відповідь обгорнута в markdown-блок, тому за вимогою «лише код» виставлено 1 бал.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і де виникла основна різниця? Найбільше набрала gemini-3.8-flash — 11/12. Найсильніше моделі розійшлися у фактичному знанні, багатокроковій логіці та якості української мови.

2. На яких завданнях оцінки двох прогонів розійшлися? За балами розійшлося лише стиснення у llama3.2:3b: перший прогін втратив частину чисел, а другий зберіг числа, але не вклався у два речення. В інших завданнях формулювання могли змінюватися, але оцінка за рубрикою залишалася тією самою.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2? Так. Усі три моделі повернули JSON, який json.loads() розібрав без попереднього очищення. При цьому локальні моделі заповнили поля неправильними значеннями, але саме формат був валідним.

4. Чи були проблеми з українською мовою? Найбільше проблем було у llama3.2:3b: змішування мов, русизми й неясні конструкції. qwen3:4b була значно кращою, але також мала русизми та неприродні фрази; gemini-3.8-flash написала найприродніше.

Обов'язковий приклад провалу

Повний текст запиту:

120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг.
Скільки рейсів? Покажи хід розв'язання.

Повний текст відповіді Пата3.2:3b (прогін 2):

Хід розв'язання:

1. Розділяємо вантаж на два частини: 45 по 8 кг і решту по 12 кг.
2. Переведемо вантажність до кілограмів: 45 по 8 кг = 360 кг, решта по 12 кг = 120 кг.
3. Сумарна вантажність: 360 кг + 120 кг = 480 кг.
4. Вантажівка тримає 1200 кг, що означає, що вона може перевезти 1200 кг.

Тепер, ми хотіли знати, скільки рейсів необхідних для перевезення вантажівки. Для цього потрібно подивитися, скільки ящиків перевозять по 8 кг.

5. Кількість ящиків по 8 кг: 120 ящиків / 8 кг = 15 ящиків.
6. Кількість ящиків по 12 кг: 120 ящиків - 15 ящиків = 105 ящиків.
7. Кількість рейсів для перевезення ящиків по 12 кг: 105 ящиків / 12 кг = 8,75 рейсів (приблизно 9 рейсів, оскільки ми не можемо перевезти половину рейса).
8. Кількість рейсів для перевезення ящиків по 8 кг: 15 ящиків / 8 кг = 1,875 рейса (приблизно 2 рейса, оскільки ми не можемо перевезти половину рейса).
9. Кількість рейсів для перевезення вантажівки: 1200 кг / 120 кг/рейс = 10 рейсів.

Окончателний ответ – 10.

Правильна відповідь:

$45 \times 8 = 360$ кг; $120 - 45 = 75$ ящиків; $75 \times 12 = 900$ кг; загальна маса 1260 кг. Оскільки $1260 > 1200$, потрібно 2 рейси.

Пояснення провалу:

Модель загубилася у багатокроковій арифметиці: неправильно порахувала масу решти ящиків, після чого почала ділити кількість ящиків на їхню вагу. Через ранню помилку кожен наступний крок спирався на вже неправильні числа, тому правдоподібно оформлений хід завершився відповіддю 10 рейсів замість 2.

Відповіді на питання (крок 8)

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці? Використовують автоматичні тести з exact match, unit-тести для коду, benchmark-набори, людське парне порівняння та оцінювання іншою моделлю як judge. Найнадійніше поєднувати кілька методів, а не покладатися на один бал.

2. Чому моделі часто ігнорують інструкцію не використовувати markdown при JSON? У навчальних даних код і JSON дуже часто оформлені markdown-блоками, тому модель відтворює звичний шаблон. Промпт є м'яким обмеженням, тому для надійності потрібні schema/structured output і програмна валідація.

3. Чому малі моделі гірше пишуть українською, ніж англійською? Англійських даних у навчальних корпусах значно більше, а в малій моделі менше параметрів для представлення рідших мовних закономірностей. Тому частіше з'являються кальки, русизми й змішування мов.

4. Чому однакова модель може дати різну якість на повторному запуску? Під час генерації використовується вибір із розподілу ймовірностей, тому навіть однаковий prompt може піти різними траєкторіями. На результат також можуть впливати технічні деталі обчислень і серверного інференсу.

5. Що таке галюцинація і чому модель радше вигадас факт? Галюцинація — це правдоподібна, але фактично неправильна відповідь. Базова LLM оптимізована на продовження тексту, а не на перевірку факту в базі даних, тому без зовнішнього джерела може впевнено згенерувати найбільш імовірний варіант.

Висновок за завданням 5

Рубрика виявилася корисною, бо розділила якість на конкретні перевірювані критерії. Найпоказовішими стали факт, багатокрокова логіка та українська мова: саме тут різниця між моделями була найбільшою. Для українськомовного тексту з трьох протестованих моделей я обрав би gemini-3.8-flash, якщо передача даних у хмару допустима.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1 / 5	0 / 5

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.7	5 / 5	—
temperature	1.5	5 / 5	0 / 5
top_p	0.1	1 / 5	—
top_p	0.5	5 / 5	—
top_p	1.0	5 / 5	—

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові	так
Отримана відповідь	1927 (неправильна)

Правильний рік відкриття Київського метрополітену — 1960. При temperature=0.0 локальна llama3.2:3b десять разів повторила однакову, але неправильну відповідь 1927, тому детермінованість не означає правильність.

Відповіді на питання (крок 7)

1. Яку температуру поставити агенту, який має повертати JSON? Я б використовував temperature=0 або дуже низьке значення, щоб зменшити випадковість. Але формат і зміст все одно потрібно перевіряти схемою, бо в моєму експерименті temperature=0 стабільно повторювала неправильний факт.

2. Яку температуру поставити генератору заголовків? Для творчого генератора я б почав приблизно з 0,7. У моєму тесті це вже дало 5 різних варіантів із 5, тоді як 1,5 часто породжувала безглузді фрази.

3. Чим top_p відрізняється від temperature за характером впливу? Temperature змінює співвідношення ймовірностей між кандидатами, роблячи розподіл гострішим або пласкішим. top_p відсікає малоймовірний хвіст і дозволяє вибір лише з мінімального набору токенів із заданою сумарною ймовірністю.

4. Чи означає $temperature=0$ повну відтворюваність? У моїх 10 запусках відповіді були однаковими: $unique=1/10$. Це демонструє високу відтворюваність у конкретному запуску, але не є математичною гарантією для всіх систем і середовищ.

5. Найбезглуздіша відповідь при $temperature=1.5$ і чому вона з'явилася? Серед відповідей була фактично беззмістовна фраза «Щенки.» як назва застосунку нагадувань про ліки. Висока температура збільшує шанс вибору малоімовірних токенів, тому зв'язок із початковим запитом може швидко втрачатися.

Відповіді на питання (крок 8)

1. Що математично робить $temperature$? Вона масштабує логіти перед softmax: нижче значення робить розподіл різкішим, вище — більш пласким. Тому при високій температурі зростає шанс вибору менш імовірних токенів.

2. Чим top_p відрізняється від $temperature$? top_p не змінює самі відносні ймовірності всіх токенів, а формує nucleus — найменший набір кандидатів, сума ймовірностей якого досягає p . Усе поза цим набором відкидається.

3. Чи є сенс змінювати обидва параметри одночасно? Можна, але тоді важче зрозуміти, який параметр спричинив зміну поведінки. На практиці зручніше зафіксувати один і налаштувати інший під конкретну задачу.

4. Чому навіть $temperature=0$ не гарантує абсолютної однаковості? На результат можуть впливати недетерміновані GPU-операції, паралелізація, зміни реалізації сервера або версії моделі. У моєму локальному експерименті 10 відповідей збіглися, але це емпіричний результат, а не гарантія.

5. Які значення $temperature$ використовують для структурованих даних і творчого тексту? Для структурованих даних зазвичай використовують 0 або низькі значення, а для творчих задач — помірно вищі, наприклад близько 0,7. Конкретне значення потрібно перевіряти на своїх даних і метриках.

Висновок за завданням 6

Експеримент показав, що temperature і top_p керують різноманітністю, а не фактичною правильністю. Для наступних робіт зі структурованими даними доцільно використовувати низьку temperature та обов’язкову програмну валідацію; значення за замовчуванням 0,7 може додавати непотрібну випадковість.

Підсумкове порівняння моделей

Таблиця 18 – Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В)	Хмарна
Швидкість генерації	59,7–140,9 симв./с у benchmark	17,022 ток./с на фактичному запиті; міркування повільне	21,2–391,4 симв./с залежно від промпту
TTFT	0,035–0,064 с після прогріву	окремо не вимірювався	1,274–5,024 с
Якість, балів /12	4 / 12	9 / 12	11 / 12
Стабільність формату	JSON парситься, але дані помилкові	JSON парситься, значення нестабільні	JSON парситься стабільно
Потрібні ресурси	локальні CPU/RAM	більше локальної RAM/CPU	обчислення на сервері провайдера
Вартість	без API-витрат	без API-витрат	безкоштовний тариф з лімітами / платний тариф
Куди йдуть дані	залишаються на пристрої	залишаються на пристрої	передаються провайдеру
Працює без інтернету	так	так	ні
Коли обирати саме її	Коли важливі приватність, офлайн-робота і мінімальна затримка на простих запитах.	Коли потрібне краще міркування локально і допустимий більший час/ресурси.	Коли пріоритетом є якість, українська мова та швидка генерація довгих відповідей, а передача даних назовні допустима.

Загальні висновки

Лабораторна робота показала, що локальні та хмарні LLM можна підключати через один OpenAI-сумісний інтерфейс, але їхня поведінка суттєво відрізняється. llama3.2:3b мала найменший TTFT після прогріву та працює повністю локально, проте часто помилялася у фактах, логіці й українській мові. qwen3:4b краще впоралася з багатокроковими задачами, але витратила значно більше токенів і часу через внутрішнє міркування. gemini-3.8-flash дала найвищу оцінку якості та найкращу швидкість генерації на довгому запиті, але залежить від мережі й передає дані провайдеру.

Окремо видно, що низька temperature робить відповіді стабільнішими, але не робить їх правильними: llama3.2:3b десять разів однаково назвала неправильний рік 1927. Тому для реальних застосунків потрібні не лише промпти й параметри генерації, а й перевірка формату, авторитетні джерела для фактів, повторні вимірювання та чітке правило вибору моделі під конкретну задачу.

Використані джерела

1. Офіційний портал Києва. «Сьогодні столичному метрополітену виповнилось 60 років!».

[Посилання](#)

2. Офіційний портал Києва. Дані про першу ділянку Святошинсько-Броварської лінії.

[Посилання](#)